

RAG Pipeline

1. Overview

This internal legal and regulatory assistant answers Armenian and English questions about the Law of the Republic of Armenia on Electronic Communications. Its evidence source is a locally processed Armenian law from ARLIS. Retrieval supplies article text to a selected generation provider, which is instructed to answer from that evidence, include article citations, and abstain when the context is insufficient.

The implementation uses ordinary Python functions, a local embedding model and FAISS index, provider SDKs, and a Streamlit interface. It does not use an orchestration framework. Source preparation is separate from answering: ingestion, chunking, and indexing produce reusable local artifacts before the application serves questions.

User question

- > meta-question check -> local description, when matched
 - > conditional query preprocessing
 - > multilingual query embedding
 - > FAISS search -> top 3 chunks
 - > context assembly and allowed article citations
 - > selected LLM provider, with original question
 - > structured answer and citations
 - > validation -> displayed response or error
-

The main entry point is `src/rag.py:ask`. `app.py` exposes question answering and a saved benchmark comparison. These are evidence-assisted answers, not a guarantee of legal correctness.

2. Source ingestion

`src/ingestion.py` fetches the ARLIS Armenian law page identified by act 1869 through `requests`, with a 30-second timeout and HTTP error checking. It decodes UTF-8 content and uses `BeautifulSoup` to select the law-body container rather than parse the entire website as legal text.

The parser identifies article headings in tables, extracts their numbers and titles, and replaces those tables with article-boundary markers. It removes navigation and executable or styling elements, excludes the presidential signature table, and treats chapter and section headings as boundaries. Explicit paragraph separators accommodate nested paragraphs in the source HTML. Whitespace is normalized within lines while paragraph boundaries remain in article bodies.

Extraction fails on missing content, unexpected tables, duplicate article numbers, missing headings, or empty article bodies. The output, `data/processed/law_articles.json`, contains **67 articles**, each with `article_number`, `title`, and `text`. This is structured local data, not a continuously synchronized legal database; running the ingestion command fetches the configured page again.

Article structure is important because citations and reference answers use legal article numbers. Preserving those boundaries makes an answer traceable to its source and avoids arbitrary text windows mixing unrelated legal provisions. The parser is intentionally specific to the inspected ARLIS layout, which also means website changes may require maintenance.

3. Chunking strategy

`src/chunking.py` creates **83 chunks** in `data/processed/law_chunks.json`. Every chunk belongs to one article and retains its number, title, a stable article-based chunk identifier, and a zero-based chunk index. There is no cross-article chunking and **no overlap**. Concatenating an article's chunk texts must reconstruct its

processed body exactly; the chunker checks this invariant.

The splitter first prefers numbered parts, then numbered clauses, and otherwise uses paragraph boundaries. Oversized units fall back to paragraphs; oversized paragraphs split at whitespace, or at the character limit when no whitespace is available. Pieces are packed together without exceeding the configured body-text limit.

The default limit is 700 approximate tokens, calculated as one token per four Unicode characters, equivalent to 2,800 characters of body text. This is a sizing heuristic, not a tokenizer count, and can underestimate Armenian token usage. Article headers added for embedding are outside that body limit. The embedding stage separately checks the actual model tokenizer limit.

The choice balances preserving conditions and exceptions against focused retrieval and controlled prompt size. Omitting overlap avoids duplicate evidence in this small corpus, and the saved evaluation supports adequate article coverage. It does not establish that every relevant clause is retrieved. A condition split from its exception can still be missed, so this strategy is not assumed optimal for larger legal collections.

4. Embeddings

`src/embeddings.py` loads BAAI/bge-m3 through Sentence Transformers. A multilingual model provides one shared embedding space for Armenian source text and Armenian or English questions, avoiding separate indexes and language-specific retrieval paths. English queries are embedded directly; the pipeline does not translate them into Armenian.

Each document embedding includes the article number, title, and chunk body. The stored index metadata verifies a dimension of **1,024**. Encoding produces normalized, contiguous float32 vectors; the code checks their shape, finiteness, and unit norms. It tokenizes inputs without truncation first and rejects text exceeding the model's sequence limit rather than silently dropping content.

Document embeddings are computed locally during index construction and persisted in the FAISS index. The model files use `data/raw/models` as their local cache directory, and the loaded model instance is cached within the Python process. Runtime answering computes a new query vector while reusing the saved document vectors.

A separate English-only embedding model was not added. The shared model already recovers the required articles within three chunks on the saved benchmark, so a second embedding pipeline would add complexity without an established retrieval need. No English-only embedding comparison is claimed. Retrieval coverage and generation correctness are evaluated separately; bilingual or translated representations remain possible future work if a larger corpus exposes English retrieval weaknesses.

5. Vector search and FAISS

The index is a local **FAISS** `IndexFlatIP`, built in `src/embeddings.py:build_index`. It performs exact inner-product search. Because both document and query vectors are normalized, inner product corresponds to cosine similarity. With only 83 vectors, approximate search and a remotely managed vector service are unnecessary for this implementation.

The files `law_chunks.faiss` and `law_index_metadata.json` persist the index and its mapping to chunks. Metadata records the model name, dimension, chunk order, and SHA-256 hashes of the chunk source and index. `src/retrieval.py:load_index` checks these associations, vector count, inner-product metric, and chunk identifiers before searching. Mismatches request a rebuild rather than risk attaching the wrong article to a result.

FAISS keeps deployment simple for one mostly static legal source. The current application has no requirement for tenant isolation, complex metadata filtering, or distributed persistence. A vector database such as Qdrant, Chroma, or Weaviate would become worth evaluating with many documents, frequent

updates, filtering, multi-tenant access, distributed deployment, or larger concurrent workloads. Those needs involve operational management beyond the present local-file design.

6. Query preprocessing

Meta questions

`src/query_preprocessing.py:meta_answer` recognizes a small set of identity and capability questions, including "Who are you?" and "What can you do?". Matching normalizes case, whitespace, and punctuation, including Armenian question and emphasis marks. Language selection uses the presence of Armenian letters, not a general language-identification model.

A match returns a fixed description in Armenian or English. `ask` performs this check before provider construction or retrieval, returning provider `local`, model `meta`, and empty citation and retrieval lists. This avoids embedding work and external API calls, and avoids treating assistant-identity questions as unsupported legal questions. Unrecognized paraphrases continue through normal retrieval.

Conditional domain disambiguation

The retrieval query receives electronic-communications context only when it matches a short definition pattern and a configured ambiguous term. Examples include "Who is operator?" and "What is interconnection?". The rule allows at most six normalized words and recognizes selected forms of operator, interconnection, subscriber, and service provider in both languages.

It appends domain context to those queries; it does not prefix every question. Detailed questions and nonmatching terms receive only whitespace normalization. The augmented query is used exclusively for retrieval, while the original question remains in the generation request and returned record.

The development rationale was to address ambiguous short terms without disturbing already specific questions. An earlier unconditional-prefix approach reportedly regressed detailed benchmark retrieval, motivating the conservative rule. The current saved retrieval results verify restored benchmark coverage; they do not retain a quantitative before/after comparison for short queries. This is a narrow deterministic heuristic, not a general intent classifier. `src/evaluation/retrieval_eval.py` uses the same `retrieval_query` function as runtime RAG.

7. Retrieval

`src/rag.py:ask` explicitly calls `retrieve(..., top_k=3)`. `src/retrieval.py` loads and validates the index, embeds the query, and returns the three highest-scoring chunks with rank, score, chunk identifier, article number, title, and text. Multiple chunks may belong to the same article; three chunks do not necessarily mean three distinct articles.

The standalone retrieval function and its CLI retain a default of five, but runtime RAG and the current retrieval evaluator explicitly request three. No lexical retrieval, reranker, score threshold, or automatic query expansion beyond the conditional rule is applied.

Three chunks keep the evidence budget small while recovering required benchmark articles. This reduces prompt content relative to larger retrieval sets and is intended to limit irrelevant context and generation overhead; no controlled latency improvement from changing this parameter is claimed.

The saved `data/evaluation/retrieval_results.json` matches the final 18 question IDs, wording, languages, types, and article mappings. Its metrics exclude the three unanswerable questions and average over 15 answerable or synthesis questions:

Metric	Saved value	Interpretation
Recall@1	0.7667	Mean fraction of expected articles found at the first unique-article rank.
Recall@3	1.0000	All mapped articles were recovered within the retrieved three chunks.
MRR	0.9222	Mean reciprocal rank of the first expected article, using unique-article ranks.

All three synthesis questions also have full expected-article coverage. The saved synthesis field retains the legacy name `average_synthesis_coverage_at_5`, although each saved result contains only three chunks. It should not be interpreted as evidence of a separate five-chunk experiment. These results establish coverage on a small, development-used dataset, not universal retrieval accuracy or complete clause-level coverage.

8. Context assembly

`src/rag.py:assemble_context` sorts results by retrieval rank and serializes each with an article label, title, chunk identifier, and body text. Separate chunks remain separate context blocks. The orchestrator creates a deduplicated `allowed_citations` list from their article numbers.

Each adapter receives that context, the original question, and the allowed list. The retrieved chunks are the only legal evidence supplied to generation. Articles merely mentioned within a retrieved provision do not automatically become citation candidates. This restriction prevents citing an unseen article just because its number occurs in the text, but it does not prove the answer's claims are supported.

9. Answer generation

All four adapters implement the same `generate(question, context, system_instruction, allowed_articles)` interface and return the shared `ProviderResponse`. The completed comparison includes Gemini, Groq, and Mistral as the three required free-tier providers, plus OpenAI as an additional paid comparison. Model names and API keys are read from environment variables or `.env`; selecting a provider does not change retrieval.

The shared instruction requires use of supplied context only, a response in the question's language, concise but complete legal information, and abstention when evidence is insufficient. It treats question and context as data rather than instructions overriding the grounding policy. Article references belong in the citations list rather than answer prose.

The output contract is exactly:

```
{"answer": "...", "citations": ["45"]}
```

Gemini uses its Interactions API with a JSON response schema. Mistral uses streaming chat with a strict JSON schema. OpenAI uses the Responses API with a strict JSON schema. Their citation schemas restrict values to allowed article numbers. Groq uses streaming JSON-object mode and local validation rather than the same server-side schema constraint. These differences are relevant implementation details when interpreting provider comparisons.

All adapters accumulate streamed generated text before JSON parsing. They measure request-to-first-nonempty-output-text time and request-to-stream-completion latency with a monotonic performance timer. Empty initial events do not count as output. TTFT may begin with JSON syntax rather than the first word of the natural-language answer. Timing excludes retrieval, client setup, and later parsing, so it is not end-to-end application latency. Token usage comes from provider metadata, without manual estimation.

The Streamlit interface waits for the completed result rather than displaying incremental tokens. Grounding and language requirements remain instructions; models can still omit essential information or produce unsupported claims.

10. Citation validation and grounding

`src/providers/base.py:validate_answer` requires an object containing exactly `answer` and `citations`, a nonempty answer string, and a list of nonempty citation strings. Adapters parse accumulated JSON and check completion status; malformed output, incomplete generation, and provider errors produce failed responses. The shared orchestrator checks that every citation belongs to the retrieved article set. Invalid answers are withheld, and available raw answer/citation fields support diagnosis.

These checks do not assess whether a cited article entails a claim, enforce the answer's language, detect every citation marker inside prose, or establish that an empty citation list represents a valid abstention. There is no semantic claim verifier in the runtime pipeline.

Evaluation therefore distinguishes three questions. Deterministic citation precision, recall, and F1 compare returned article sets with expected article sets. Human answer accuracy checks essential legal facts against semantic reference answers. Human hallucination review checks substantive claims against the actual retrieved context. A correct citation can accompany an incomplete answer, and a legally true statement can still be unsupported by the supplied chunks.

11. Abstention behavior

When context does not support an answer, the instruction asks the model to state that the provided law contains insufficient information and return an empty citation list. The benchmark includes employee income tax, corporate income tax, and a universal minimum guaranteed mobile internet download speed as unanswerable questions. The system should not fill these gaps with outside knowledge.

Abstention is model-directed, not triggered by a similarity threshold or an automatic answerability classifier. An empty citation list alone is not evidence of correct abstention. Under the evaluation rubric, unsupported substantive answers remain inaccurate and hallucinated even when their citations are empty.

The finalized summary records **100% abstention accuracy for all four providers**, based on three unanswerable questions per provider scoring 2 in human review. This is a limited observed result, not a guarantee for arbitrary out-of-scope or adversarial inputs.

12. Caching and runtime efficiency

Ingestion, article chunking, and document embedding are offline preparation steps. They are not repeated per question. `get_model` uses a one-entry process-local cache, so the embedding model is loaded once per process after first use. Model weights are also cached on disk.

The persisted FAISS index is reused, but it is **read and validated on every retrieval call** rather than cached in memory. There is no answer cache, query-embedding cache, Redis layer, or distributed cache. Streamlit retains the last completed answer in session state across reruns, which is presentation state rather than a general query cache.

Benchmark execution saves each completed attempt through a temporary file followed by replacement. Resuming skips saved successes and retries failed records. Optional pacing sleeps between actual attempts, outside provider timing. A saved rate-limit failure stops the run; there is no immediate application-level retry loop. These choices make limited API quotas easier to manage without introducing background-job infrastructure.

13. Same-pipeline benchmark design

The final benchmark has 18 questions: six Armenian answerable, six English answerable, three synthesis, and three unanswerable. Four providers produce 72 final provider-question records, paired with 72 manual reviews. `src/evaluation/benchmark.py` calls the same `rag.ask` path used by the app.

The comparison holds the question set, preprocessing, embedding model, document index, three-chunk retrieval, context construction, shared grounding instruction, and logical output format constant. Only the selected generation adapter and its model change. Providers independently receive context retrieved through that shared path, rather than provider-specific retrieval tuning.

This controls major sources of variation, but it is not a perfectly identical API experiment: schema support, decoding controls, reasoning settings, and provider infrastructure differ. Accuracy and hallucination are manually reviewed; no LLM judge defines the official results. Aggregation validates source records and joins reviews by provider and question ID.

The Benchmark tab reads the finalized summary without provider calls. Its separate optional run form executes one explicitly selected provider, preserving resume behavior. Raw responses require human review and summary regeneration before affecting the saved comparison. This keeps exploratory runs separate from finalized human judgments.

14. Key design decisions

Decision	Choice	Reason	Tradeoff
Embedding model	Multilingual BGE-M3	Shared Armenian/English semantic retrieval	Cross-language alignment can miss legal nuance.
Chunk boundaries	Article-aware, no overlap	Traceable citations and preserved source structure	Split articles can separate relevant conditions.
Vector storage	Local FAISS exact search	Small static corpus and simple deployment	Limited operational and filtering capabilities.
Context budget	Three ranked chunks	Full mapped-article coverage on this benchmark	Unseen questions may need more evidence.
Short queries	Deterministic domain disambiguation	Narrow intervention for ambiguous definitions	Vocabulary and patterns require maintenance.
Orchestration	Direct Python, no LangChain	Explicit control flow and few moving parts	Integration and validation are maintained locally.
Generation contract	JSON answer and citations	Consistent parsing across providers	Format validity does not imply factual validity.
Semantic evaluation	Human review	Assess completeness and context support directly	Small-scale and subject to reviewer judgment.

15. Known limitations

The corpus contains one Armenian law snapshot, with no automated legal-update monitoring or incremental ingestion pipeline. English questions depend on multilingual alignment. The application has no authentication or authorization layer, production vector database, or distributed job management. Its local files and blocking execution are appropriate to a take-home demonstration, not evidence of production readiness.

The benchmark is small and was used during retrieval development; it is not a broad held-out generalization test. Article-level recall can hide missing passages within a split article. Some question-to-article mappings have scope ambiguities recorded in the evaluation rubric. Generation completeness varies

by provider, and the saved question-type results show lower accuracy for synthesis than for either answerable language group across all four providers.

There is no hybrid lexical search, reranker, automatic provider fallback, semantic citation verification, or comprehensive prompt-injection defense. Grounding instructions and citation membership checks reduce some failure modes but do not guarantee legally sound answers. Users must review evidence and applicable law before relying on an answer for a consequential decision.

16. Future improvements

A larger held-out benchmark should precede additional complexity, especially for cross-language legal terminology, multi-article synthesis, and adversarial questions. Adding laws and regulatory documents would require source/version metadata and a deliberate update and reindexing process.

Hybrid BM25 and dense retrieval, reranking, and stronger claim-to-passage checks could address demonstrated retrieval or grounding errors. Translation or dual-language representations would be justified if English retrieval becomes a measured bottleneck. A production vector database, access controls, and operational monitoring become appropriate as deployment requirements grow. Provider routing and fallback could improve availability, but should preserve transparent reporting of which model answered. None of these features is currently implemented.

17. Code map

Component	File	Responsibility
Ingestion/parser	src/ingestion.py	Fetch ARLIS HTML and extract structured articles.
Chunker	src/chunking.py	Split within articles while preserving text and metadata.
Embeddings/index builder	src/embeddings.py	Encode normalized vectors and persist FAISS plus metadata.
Retrieval	src/retrieval.py	Validate saved artifacts, embed queries, and rank chunks.
Query preprocessing	src/query_preprocessing.py	Local meta answers and conditional domain disambiguation.
RAG orchestration	src/rag.py	Select provider, retrieve three chunks, assemble evidence, validate citations.
Response contract	src/providers/base.py	Shared response fields and structured-answer validation.
Generation adapters	src/providers/gemini.py, groq.py, mistral.py, openai.py	Provider requests, streaming, timing, usage, and errors.
Benchmark runner	src/evaluation/benchmark.py	Resumable provider runs and deterministic citation metrics.
Retrieval evaluation	src/evaluation/retrieval_eval.py	Article coverage and reciprocal-rank measurements.
Dataset validation	src/evaluation/benchmark_validation.py	Validate question schema and source article existence.
Final aggregation	src/evaluation/evaluation_summary.py	Join human reviews with final results and calculate summaries.
Streamlit application	app.py	Question form, answer/context display, comparison, and explicit benchmark control.
Benchmark presentation	src/ui.py	Validate saved summary and format comparison/type tables.